

"MiniPong": Tutorial de primeros pasos para tomar contacto con los 8-bits, by GuerraTron26
Esto es un proyecto de ejemplo "miniPong" para "SMS" (Master System) desde "C" (o "asm"), lo compila y limpia, preparado para añadir código al archivo "c" y recompilar con el "compile.bat" creado.
Author: Juan José Guerra Haba - dinertron@gmail.com - May, 2026
Web: <https://guerratron.github.io/>
Categoría: Master System, 8 bits console
License: Free BSD. & Open GPL v.3. Keep credit, please.
Versión: 1.0.0
File: compile.bat -> src/MiniPong.c Main Class: N/A :: function main(void)

INTRO v2.0

MiniPong

Incluye unos **Makefile** preparados para trabajar con una estructura de carpetas conveniente; una vez creado el proyecto ejecutar `> make`, o para Windows simplemente ejecutar el **compile.bat**.

O si se desea utilizar **VSC** (Visual Script Code), también viene preparado para abrirlo como proyecto.

Este proyecto se ha construido utilizando la herramienta "**C2SMS-v2**" para "**Proyectos de C (o ASM) a SMS**".



ATENCIÓN: No voy a utilizar frameworks ni librerías externas, ni "devkitSMS", ni "CrossZGB" ni siquiera "SMSLib".

REQUISITOS

- **SDCC** instalado en el sistema y la orden **Make** para la línea de comandos.
- Para probar los juegos algún emulador como **Emulicious**.
- Herramientas para trabajar con archivos **C** o **ASM**.
- Algunas herramientas extras vendrían bien para convertir gráficos o música a arrays "c". En la carpeta **/tools** vienen algunas sugerencias.

No es obligatorio, pero sería cómodo si se tiene instalado **VSC** para la modificación del código fuente.

MiniPong: Tutorial 2 v1.0

Bueno en el primer tutorial **balln** vimos como crear un juego **sms** y como cargar una serie de "assets" en la memoria de video, todo **a pelo**.

En este otro tutorial vamos a ayudarnos del puñado de funciones de utilidad que he preparado en la herramienta **c2sms** para no tener que lidiar con registros del **z80** ni comandos del **vdw**.

Vamos a construir un jueguecillo homenaje a **pong**, pero rudimentario, con el que podamos experimentar la forma correcta de definición e importación de recursos.

Al terminar tendremos un juego con tres pantallas con distintos fondos, cargaremos sprites sencillos y otros más complejos (monotile y multitile) e incluso tendremos música "arkanoid" de fondo, efectos de sonido, control del pad, detección de colisiones y otras bondades añadidas.

PASO 1: Inicio

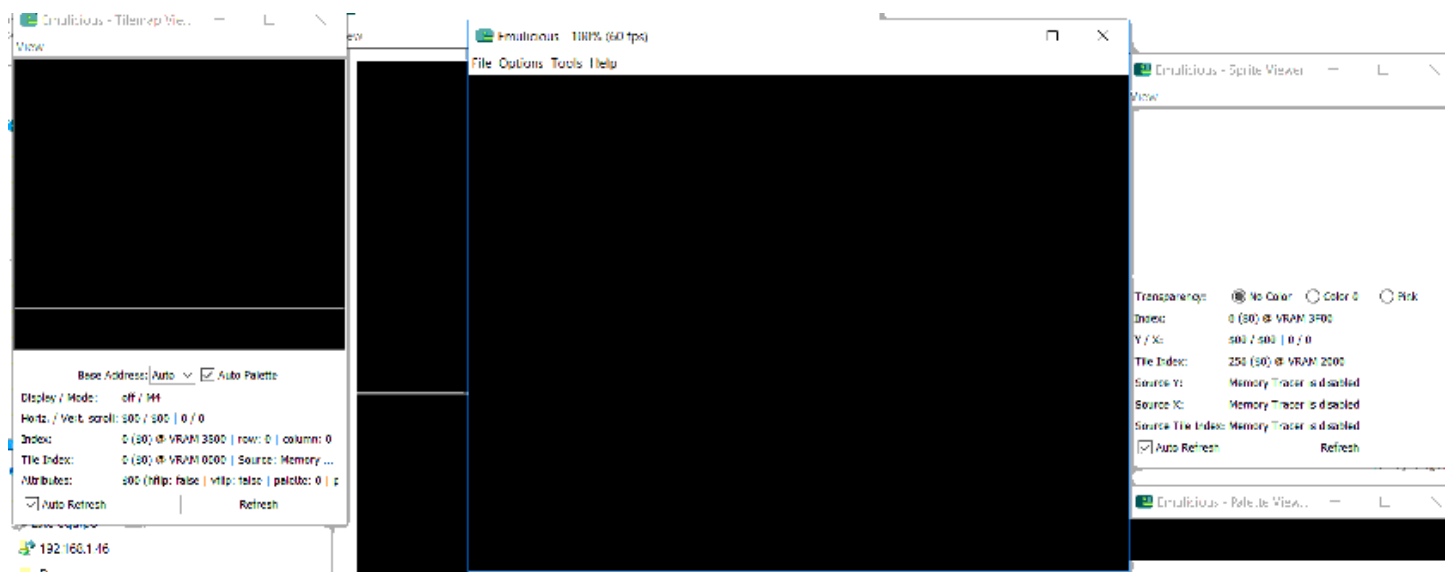
Primero construimos un proyecto vacío llamado **MiniPong** con la orden `> c2sms -e MiniPong`. Ahora copiamos los archivos que vienen en **files.zip** y los situamos en la carpeta del proyecto, cada uno en sus respectivas carpetas (sustituir si es necesario). De forma paralela podemos abrir la carpeta del proyecto en **vsc** [OPCIONAL].

En el archivo **const.h** se han definido una serie de objetos gráficos que utilizaremos en **load_commons.h** y en las restantes pantallas.

[i]

Los archivos en la carpeta **_temp** son los que tienen el código que iremos incluyendo paso a paso. De momento no afectan para la compilación, y al terminar el juego esta carpeta **puede borrarse** perfectamente.

Ya tenemos nuestra base montada, si ejecutásemos `> make valid` nos generaría un juego que en principio no hace nada hasta que no retoquemos el código fuente.



Al igual que el anterior tutorial, tenemos que ir paso a paso activando una serie de **#defines** incluidos en **const.h**, compilando y probando en **Emulicious**.

Se han preparado unos **define** para habilitar (con 1) ó deshabilitar (con 0) cada nivel de tutorial donde nos encontremos:

```

/// src/const.h
// -----
// HABILITA O DESHABILITA EL NIVEL DE TUTORIAL (EMPEZAMOS EN TUTO1 CON TODO A 0)
#define _TUTO2_COMMONS_1 0    /// cargamos un fondo
#define _TUTO2_COMMONS_2 0    /// carga sprites: pelota, raquetas, marcadores,
logo, ..
#define _TUTO2_COMMONS_3 0    /// visualizamos sprites sobre el fondo de la
pantalla

#define _TUTO2_INTRO_1 0      /// dibuja el fondo y emite un breve pitido
#define _TUTO2_INTRO_2 0      /// dibuja el objeto gráfico de las "raquetas" (es
más visible si desactivamos _TUTO2_COMMONS_3)
#define _TUTO2_INTRO_3 0      /// comprueba la pulsación de botones en el
"update"

#define _TUTO2_PLAY_1 0        /// reconoce la pantalla y la añade al array de las
existentes
#define _TUTO2_PLAY_2 0        /// control del movimiento de la bola y los
marcadores
#define _TUTO2_PLAY_3 0        /// control del movimiento del logo y de los pads
(con cursor)
#define _TUTO2_PLAY_4 0        /// control de colisiones
#define _TUTO2_PLAY_5 0        /// activa pasar de pantalla (a gameover, pero aún
no existe)

#define _TUTO2_GAMEOVER_1 0    /// reconoce la pantalla y la añade al array de las
existentes
#define _TUTO2_GAMEOVER_2 0    /// cargamos los gráficos (fondo, raquetas y logo)
#define _TUTO2_GAMEOVER_3 0    /// detección de botones para volver a pantalla
inicial
// -----

```

PASO 1.2: Comunes

[_TUTO2_COMMONS_1, _TUTO2_COMMONS_2 y _TUTO2_COMMONS_3]

Vamos a utilizar el archivo **load_commons.h** para cargar una serie de objetos gráficos comunes a todas las pantallas.

Este archivo tiene tres funciones **load_commons**, **spritesInit** y **spritesDefine**, la primera es la que se llamará desde el **main** y se encargará de llamar a las otras dos.

Al activar `#define _TUTO2_COMMONS_1 1` conseguimos cargar un fondo jugando con los dos tiles que contiene. Compilando podemos comprobar esto, un sencillo fondo sin más.

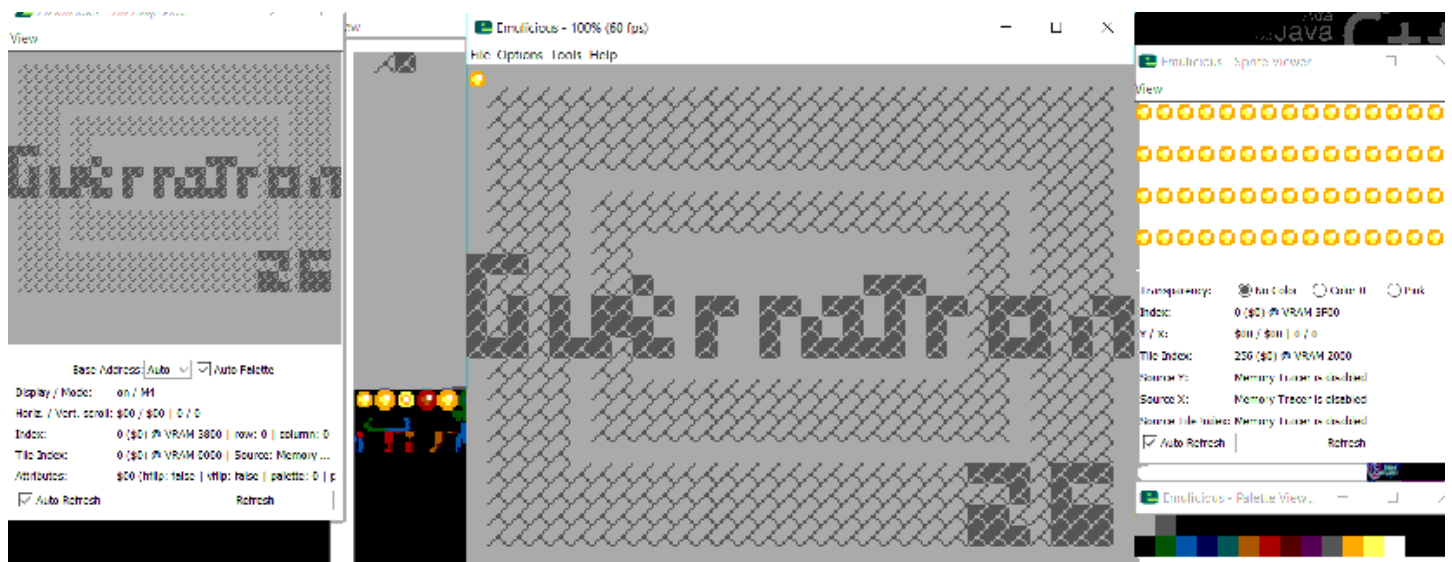


Tanto **spritesInit** como **spritesDefine** se encargan de preparar los objetos gráficos que se utilizarán como: sprites-monotile, sprites-multitile, sprites-animados, o incluso sprites como imágenes estáticas ...

En función de lo que se desee se utilizarán unas funciones u otras, pero básicamente sigo los pasos de preparar los objetos gráficos en **spritesInit** mediante:

- Defino el objeto de movimientos del sprite con `moveDef(..)`
- Defino luego el objeto gráfico completo con `_ballDef(..)`
- Cargo el sprite con `load_sprite(..)`
- Voy incrementando consecutivamente ciertas variables que afectan a la dirección en memoria del siguiente sprite a asignar (como **next_x_address**, **address_til**, **prev_tiles**, ..)

Activamos el `#define _TUTO2_COMMONS_2 1`, compilamos y pueden verse cargados los sprites que se utilizarán para el juego: pelota, raquetas, marcadores, logo, ..



Luego en **spritesDefine** los cargo en memoria con funciones como **toAnyDefine** o **toTilesifyDefineUpdate**, puede verse esto último activando el `#define _TUTO2_COMMONS_3 1` que al compilar podremos ver los sprites sobre el fondo de la pantalla.



Así que lo que hace **load_commons** es desactivar el display, limpiar la VRAM, inicializar los objetos gráficos y cargarlos en memoria, y volver a activar el display.

[i]

El archivo es lioso pero ahorra mucho código y cálculo de las direcciones a asignar a cada sprite.

Simplemente tomarlo como referencia copiando fragmentos de código para adaptarlo a cada situación.

PASO 2: Establecer Pantallas

Anteriormente dijimos que vamos a trabajar con tres pantallas. Para eso hay que definirlas lo primero en el archivo `inc/defines.h` en el "enum" **game_state_e** y luego en el archivo `src/states.h` importamos cada archivo de pantalla y le incluimos el siguiente código por cada una cargada (sustituimos [NOMBRE] por el nombre de la pantalla):

```
#include "stage_[NOMBRE].h"

//extern game_state_e game_state;
void load_[NOMBRE](uint8_t index);
game_state_e init_[NOMBRE](void);
game_state_e update_fast_[NOMBRE](uint16_t cont, uint32_t delta);
game_state_e update_[NOMBRE](uint32_t delta, uint8_t master);
game_state_e draw_[NOMBRE](uint32_t delta, uint8_t master);
```

estas funciones deben estar definidas en el archivo de cada pantalla incluida.

Y dentro de la función **loadStatesFunctions(..)** una línea por cada función de carga de pantalla:

```
..
    load_[NOMBRE](index++);
..
```

Hay que tener en cuenta que ese **index** que se le pasa debe estar en consonancia con el "enum" definido en "defines.h" **game_state_e**, y en el mismo orden.

Primero veremos como se hace esto con la pantalla de **inicio**, después la de **play** y finalmente la de **gameover**

Pantalla INTRO

[TUTO2_INTRO_1, TUTO2_INTRO_2 y TUTO2_INTRO_3]



Podemos desactivar temporalmente ****_TUTO2_COMMONS_3**** para ver los efectos en esta pantalla, y posteriormente volverlo a habilitar.

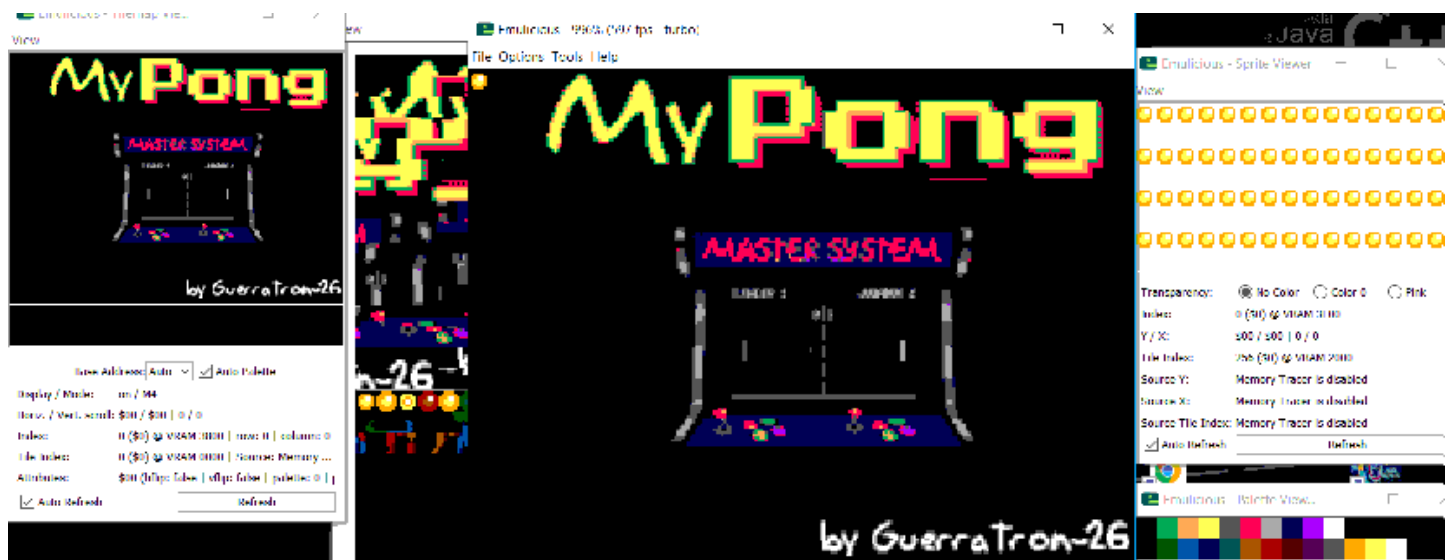
Es la más fácil, habilitamos directamente el **#define _TUTO2_INTRO_1 1** que lo que hace es habilitar el código dentro del cuerpo de la función **init_intro()** del archivo **stage_intro.h**:

```
..
__asm__("di"); //; Des-Habilitar interrupciones
// enable display (0x8160)
VDP_ADDRESS = 0b00100000; // 0x60; // 96
VDP_ADDRESS = 0x81;
// 4. Silenciar el Canal 0 // 1001 VVVV -> 1001 1111 (Volumen 15 = Silencio)
PSG = 0x9F;
// draw background image
draw_bg(_intro_pal, _intro_til, (uint16_t)_intro_size, _intro_tilemap);

// enable display (0x8160)
VDP_ADDRESS = 0b01100000; // 0x60; // 96
VDP_ADDRESS = 0x81;
// load music
playBeep(100, 2);
__asm__("ei"); //; Habilitar interrupciones
..
```

Ya tenemos otro fondo distinto de pantalla en la pantalla de inicio con un ligero pitido al iniciar.

Ejecutar > **make valid** para comprobarlo en el emulador.



Está todo comentado, la parte más importante es la carga del fondo que lo hacemos a través de la función integrada **draw_bg(..)** que le pasamos el juego de arrays definidos en el archivo **intro_def.h** que hemos importado a través de **ires.h**

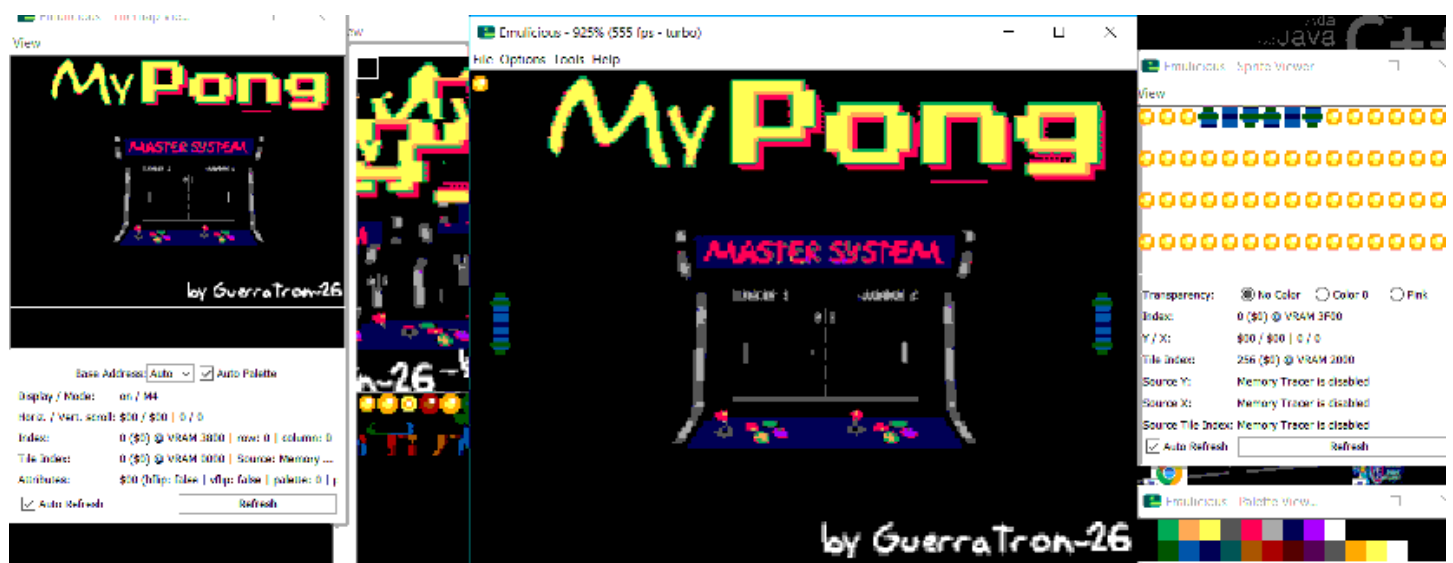
Ahora vamos al método **draw**, habilitamos **#define _TUTO2_INTRO_2 1** y nos aparece el siguiente código dentro del cuerpo de la función:

```
..
toTilesgifyDefineUpdate(&sPad1, -1, -1, -1, 0);
// toAniPos(&sPad2, -1, -1);
toTilesgifyDefineUpdate(&sPad2, -1, -1, -1, 0);
..
```

Acabamos de dibujar las "raquetas"; recordemos que son recursos comunes que ya cargamos en el **load_commons.h**, ahora los dibujamos.

Esto ya lo hicimos anteriormente de forma similar en el genérico **_TUTO2_COMMONS_3**, por eso es más evidente si probamos a desactivar primero éste.

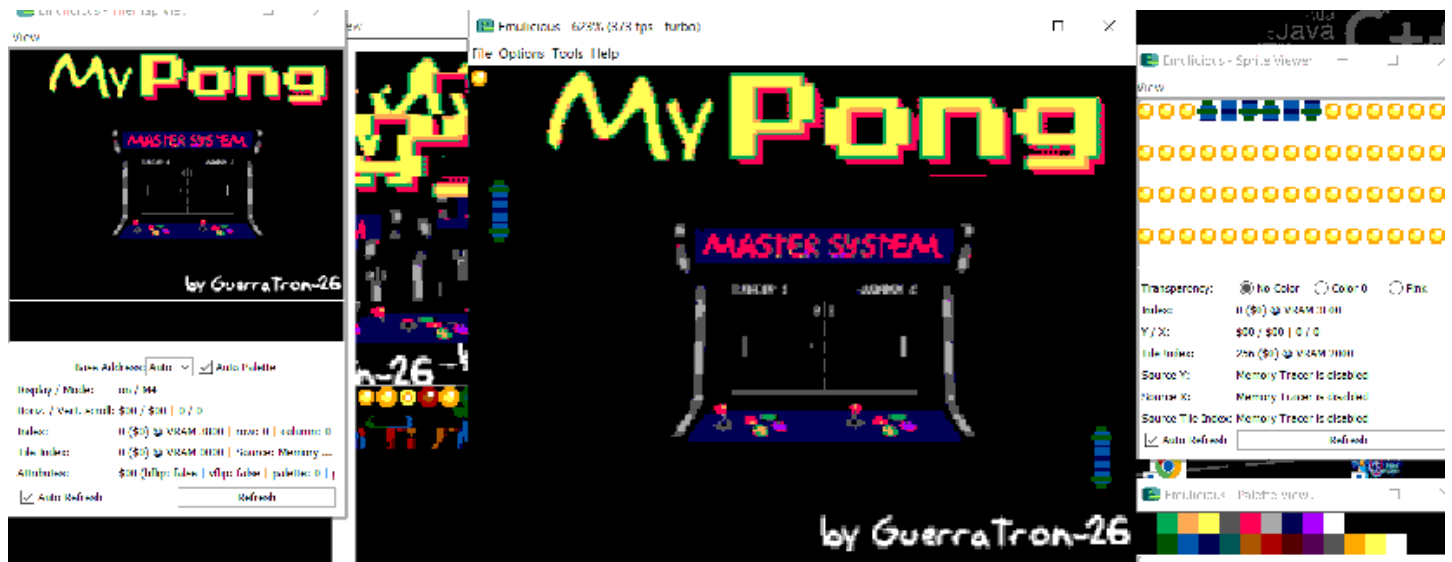
Digamos que **_TUTO2_INTRO_2** visualiza recursos que ya estaban visualizados anteriormente (no es lo más **óptimo** pero para este tutorial nos sirve).



Le toca al método **update**, habilitamos **#define _TUTO2_INTRO_3 1** y nos habilita código que lo que hace es comprobar la pulsación de botones. Puede verse el código habilitado en el cuerpo de la función.

[!]

A partir de ahora el código es mejor visualizarlo directamente en el archivo que estemos tratando ya que suele ser bastante largo para incluirlo aquí.



Si compilamos ahora podemos observar que responde a la **cruzeta del pad** moviendo las dos paletas, y también que quiere avanzar de pantalla al pulsar el botón de **START** pero no avanza a **NADA** porque aún no hemos definido más pantallas. [Será nuestro siguiente paso]

[i]

Recordar que tenemos que retornar el estado del juego en todas las funciones! y volver a activar **_TUTO2_COMMONS_3**

Pantalla PLAY

[_TUTO2_PLAY_1, _TUTO2_PLAY_2, _TUTO2_PLAY_3, _TUTO2_PLAY_4 y _TUTO2_PLAY_5]

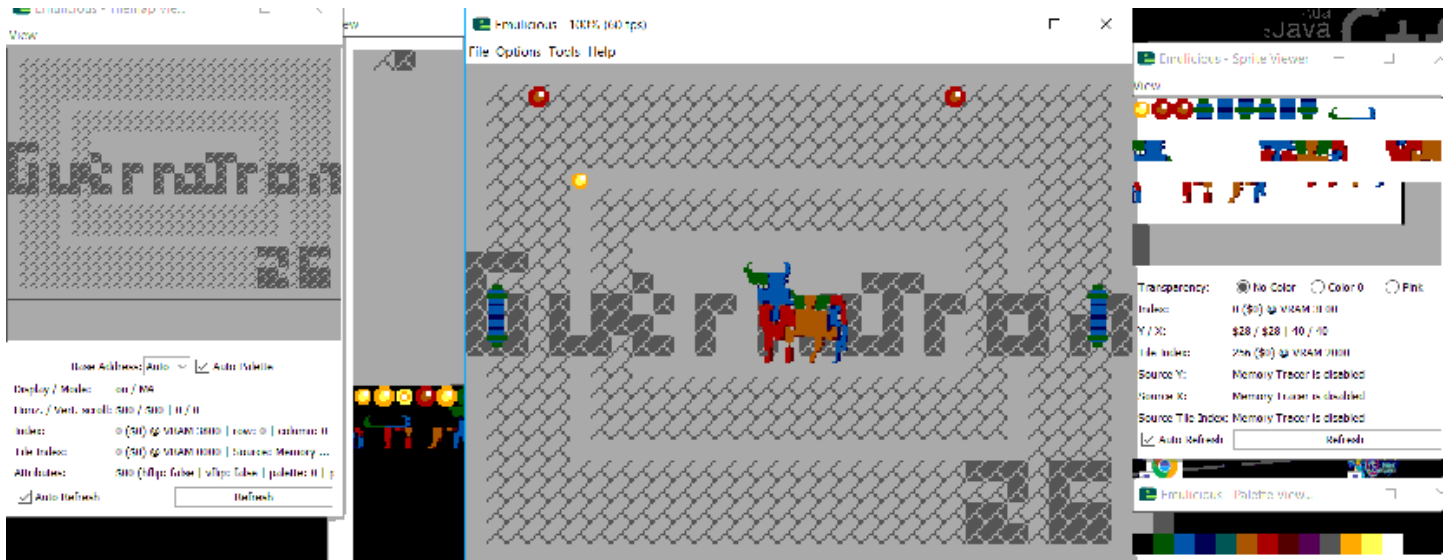
La pantalla más importante, contiene toda la lógica del juego.

A pesar de ser un juego pequeño tiene bastante código y lo mejor es copiar el archivo directamente desde la carpeta **_temp/src/stage_play.h** hasta **src/stage_play.h**. También tenemos que sobrescribir desde **_temp/src/states.h** hasta **src/states.h**

Si analizas el juego puede observarse que sigue el mismo patrón que la pantalla anterior (carga de gráficos y definición de sprites) pero además utiliza detección de colisiones entre sprites y control de pulsaciones del pad. Además controla la vida de cada jugador **lives1** y **lives2** para saber quien gana estableciendo las variables **win1** y **win2** y pasar a la siguiente pantalla.

Pero ahora debemos activar **#define _TUTO2_PLAY_1 1** para que el archivo **states.h** reconozca la pantalla y la añada al array de las existentes **sFs**.

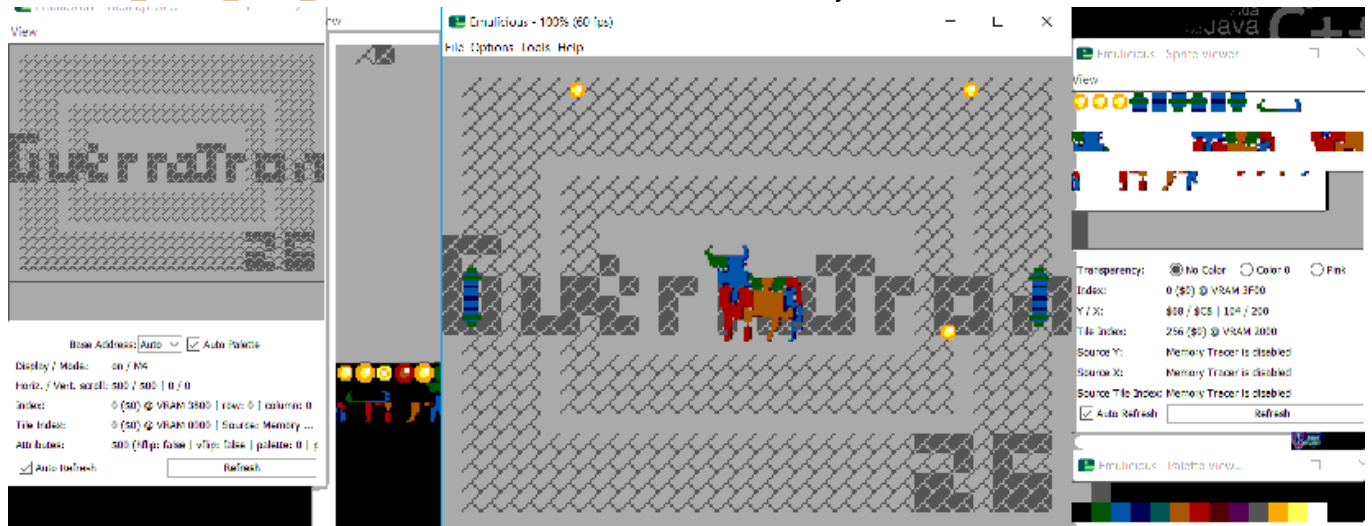
compilar con **> make valid** y probar en emulador.



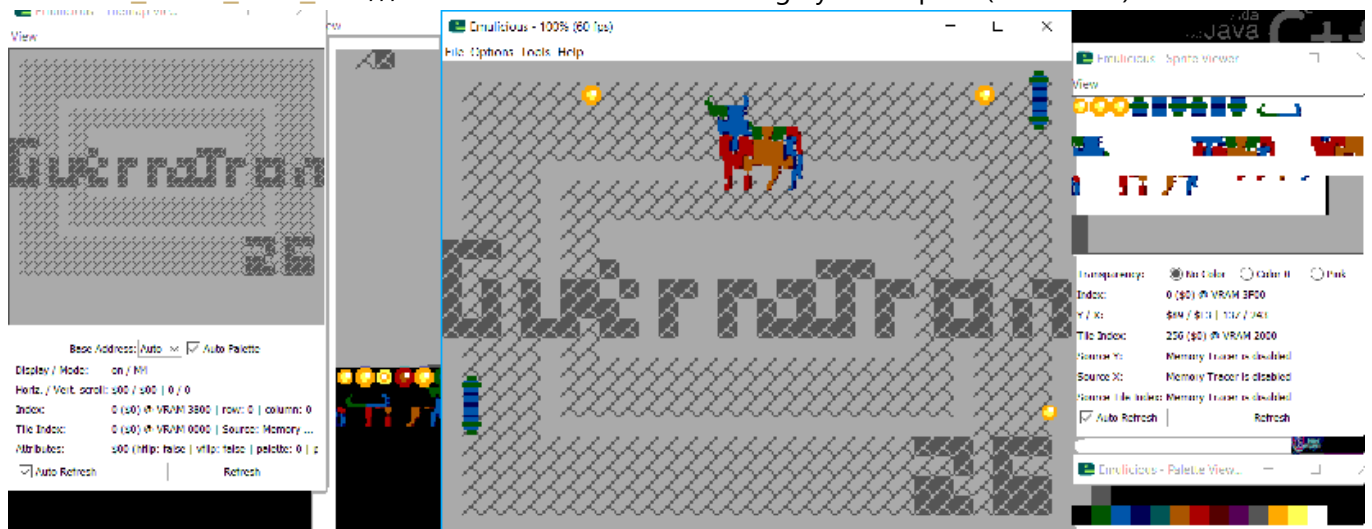
Ahora tenemos todos los gráficos cargados: fondo, bola, raquetas, marcadores y logo además de la música de fondo. Pero no avanza y no detecta pulsaciones de botones; **nos falta darle vida !**.

Los siguientes pasos actúan sobre el **update** y serían:

- `#define _TUT02_PLAY_2 1` /// control del movimiento de la bola y los marcadores



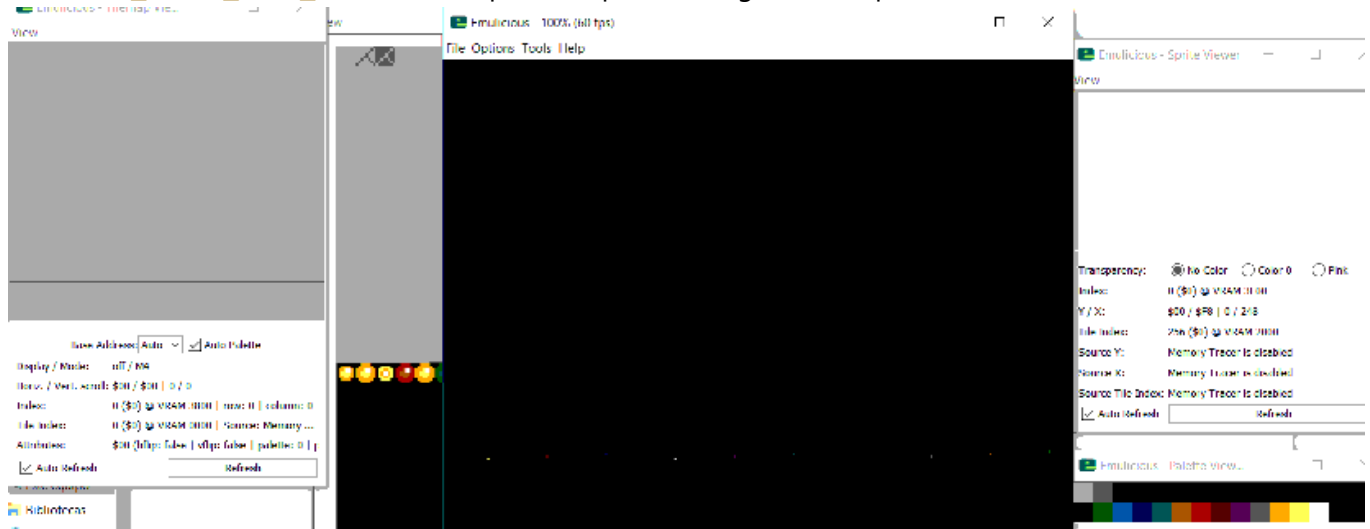
- `#define _TUT02_PLAY_3 1` /// control del movimiento del logo y de los pads (con cursor)



- `#define _TUT02_PLAY_4 1` /// control de colisiones



- `#define _TUT02_PLAY_5 1` /// activa pasar de pantalla (a gameover, pero aún no existe)



Se aconseja ir activando y compilando al mismo tiempo para probar uno a uno.

Habrás observado que al activar el último *define* y al consumir todas las bolas del marcador (bola roja) intenta pasar de pantalla pero no puede ya que aún no hemos implementado la pantalla **gameover**, y por lo tanto regresa a la pantalla inicial.

VAMOS A SOLUCIONARLO !

Pantalla GAMEOVER

[TUTO2_GAMEOVER_1 y TUTO2_GAMEOVER_2]

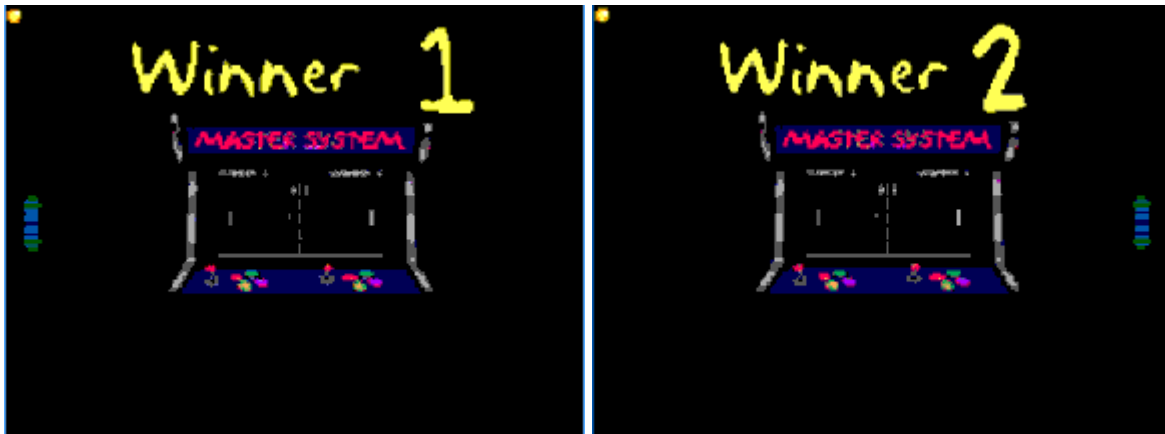
Pantalla final, muestra el ganador en base a las variables establecidas **win1** y **win2** y permite el movimiento de su raqueta. Pulsando un botón activo volvemos desde esta a la pantalla de inicio.

Al igual que el anterior primero activamos `#define TUTO2_GAMEOVER_1 1` para que el archivo **states.h** reconozca la pantalla y la añada al array de las existentes **sFs**.

Seguidamente copiar el archivo directamente desde la carpeta **_temp/src/stage_gameover.h** hasta **src/stage_gameover.h**.

Activando `#define TUTO2_GAMEOVER_2 1` cargamos los gráficos (fondo, raquetas y logo), ejecutar `> make valid` y probar, se verán los gráficos e indicará el jugador que ha ganado (1 o 2) pero aún no se puede avanzar de pantalla.

Si activamos el último `#define TUTO2_GAMEOVER_3 1` permitimos la detección de pulsación de botones que nos permiten volver a la pantalla inicial.



FIN

Ya hemos finalizado este tutorial, he preferido realizarlo por pasos de activación para que se vea qué es lo que hace cada parte de código. Seguramente lo más lioso sea la parte inicial de la carga común de recursos en **load_commons.h** pero es cuestión de práctica y pensar que son pasos cuya mecánica se repetirá seguramente en cualquier tipo de juego.

También el control de la lógica del juego en **stage_play.h** puede resultar confuso (y seguramente muuuuuuy mejorable), pero nos ha funcionado.

Hemos completado con este tutorial las partes básicas de cualquier videojuego, lo hemos programado siguiendo el ciclo **init-update-draw**, utilizamos control de botoneras, colisiones, movimientos, creación de pantallas, música de fondo he incluso algún efecto sonoro (también visual si activamos las animaciones) y todo de una forma mucho más sencilla que accediendo directamente a los registros y comandos del procesador de vídeo. **QUÉ MÁS SE PUEDE PEDIR!**

Es un juego pequeño de 32 Kb y sencillo (y no muy jugable) pero funciona y es el primer paso para seguir programando para la Master System. Además hay que tener en cuenta que no utilizamos ninguna librería de soporte.

Puede obtenerse más información generando y mirando la documentación. Muchas Gracias por interesarte en esta herramienta y seguir los tutoriales, espero que le sirvan de utilidad.

USO RÁPIDO

Después de modificar el código **C** ir a la ventana de comandos y ...

> **make**

... voilà ! ya tenemos nuestro archivo **.sms** en el directorio **"bin"**; incluso puede probarse el **"sms"** generado en un emulador como *"emullicius"*.

... el siguiente pasito es mejorar el código **"c"** (o los valientes el **"asm"**) y *"re-compile"*

EXTRAS

Mirar en la carpeta "tools", en mi pc tengo enlaces a pequeñas herramientas, pero evidentemente cada cual incluirá aquí lo que prefiera.

Por ejemeplo se aporta algún enlace a herramientas externas como **Img2SMS** o **TilesZipViewer** que nos ayudan con la tarea de convertir imágenes a un formato compatible con la Master System. O **retilesgif** que ayuda a desplazar los tilesets con el color de la paleta seleccionada. Muy importante también **checksumfix** que repara la firma del checksum en el sms.

AGRADECIMIENTOS

Antes de nada mi reconocimiento y Mil-Gracias a todos los *Masters* del código que nos han ayudado con sus útiles herramientas y sus conocimientos, en particular **Maxim, Bock, Avelino Herrera, Calindro, sverx, SteveProXNA, toxa, kusfo ...** y otros mil maestros más. Y por supuesto también a webs como "<https://www.smspover.org/>", "<https://avelinoherrera.com/blog/index.php?m=06&y=25>", "<https://raelcunha.com/2016/07/29/my-retro-journey-sega-master-system/>", "<https://steveproxna.blogspot.com/2017/09/devkitsms-programming-setup.html>", "https://www.elotrolado.net/hilo_tutorial-muy-basico-de-c-para-master-system_2208028" y todos los sitios donde he leído tutoriales o he aprendido algo.

QUE QUEDE CLARO !!!

- Esto **ES** un tutorial a título orientativo (para tomar contacto) de cómo desarrollar un juego o app sencillo para la *Master System I*, sin dependencias externas y con las herramientas que utilizo YO. Simplemente por si puede servirle a alguien de ayuda.
- Esto **NO** es la mejor manera de crear estos juegos, existen herramientas especializadas infinitamente mejores como **devKitSMS, SMSLib, CrossZGB, GBDK-2020**, ... y seguramente muchas más, de gente que sabe **muchiiiiisimo** más que yo sobre este tema. Yo sólo soy un *aprendiz de brujo* que disfruto programando en mis ratos libres.
- Esto **NO** es un tutorial para aprender "buenas técnicas de programación en **C** ni **ASM**", ni cómo utilizar los mejores estándares y paradigmas de programación, ni tampoco programación avanzada.
- Esto **NO** es un tutorial para aprender todas las características técnicas de una máquina como la **SMS**. Tampoco sabría dar detalles sobre el **start-up** de la ROM, ni muchos otros tecnicismos.

Para profundizar en conocimientos técnicos sobre la forma correcta de programación en estas videoconsolas de 8 Bits sería mucho mejor seguir tutoriales de gente como la mencionada al principio.



YO **NO** soy ningún experto, como los mencionados anteriormente (OJALÁ algún día!)

De hecho asumo que puedo estar equivocado en las conclusiones que he ido sacando de mis experiencias con la programación y que algunas explicaciones técnicas quizás no se ajusten completamente a la realidad, en tal caso **LO SIENTO MUCHO!**



¡ Yo sólo sé QUE NO SÉ NADA !

hecho con amor por GuerraTron26. 😊 dinertron@gmail.com